

Informe Arquitectónico Completo: Base de Datos y Backend

A continuación se presenta un informe **profundamente detallado y exhaustivo** sobre cómo funciona la relación entre la base de datos (Supabase), los metrados, las partidas y todo el ecosistema del backend de tu proyecto.

1. Arquitectura de la Base de Datos (Supabase)

Tu backend está soportado por PostgreSQL a través de **Supabase**, utilizando un esquema estructurado de bases de datos relacionales con llaves foráneas y vistas.

1.1. Tablas Nucleares (El Corazón del Sistema)

El sistema opera principalmente en base a **dos grandes mundos** que se entrelazan: El Catálogo (lo planificado) y Los Metrados (lo ejecutado).

A) catalogo_partidas (La Biblia del Presupuesto)

Esta tabla almacena **todas** las partidas oficiales del proyecto base.

- **Columnas Clave:**
 - id (UUID): Identificador único global de la partida.
 - codigo (text): Código técnico (Ej. OE.1.1.2).
 - descripcion, unidad: Textos descriptivos.
 - jerarquia, nivel_jerarquia, parent_id: Estructura de árbol lógico para agrupar títulos y subtítulos.
 - tipo_metrado: Define qué **fórmula matemática** usar (ESTANDAR, ACERO, HVAC_DUCTO, etc.).
 - proyecto_id: Llave foránea hacia la tabla proyectos (Hospital vs. Contingencia).
- **Datos Financieros (V21/V25):**
Almacena precio_unitario, metrado_programado, valorizacion_anterior, se_valoriza, etc.

NOTE

Las Partidas Creadas o Personalizadas (PC) antes vivían en su propia tabla (partidas_personalizadas). Sin embargo, arquitectónicamente, se están unificando en una tabla central de partidas usando el flag modificacion = 'PC' para poder indexarlas junto al catálogo oficial.

B) metrados (El Registro Vivo de la Obra)

Cada vez que un ingeniero registra un avance diario en campo, se crea un registro en esta tabla.

- **Columnas Clave:**
 - id (UUID): Identificador único del metrado.

- fecha, frente, bloque, nivel: Metadata de la ubicación exacta.
- **Vínculo con el Catálogo:**
 - partida_id: Llave foránea a catalogo_partidas.
 - codigo_partida, descripcion_partida: Se guarda en texto plano por historial (desnormalizado intencionalmente por si la partida original cambia en el futuro).
- **Parámetros de Cálculo Matemático:**
 - cantidad, longitud_area, ancho_empalme, altura_gancho, diametro, hvac_factor, nro_veces.
- **Resultados Finales:**
 - parcial y total (Calculados por el Frontend y guardados en la base de datos).
 - autor_usuario: Vinculado a ecosistema_usuarios para trazabilidad de auditoría.
 - especialidad: (Estructuras, Arquitectura, Instalaciones, etc.).

1.2. Tablas Auxiliares y Relacionales

Para evitar la repetición de datos, tu sistema usa tablas pivote e independientes:

- **personal:** Contiene la lista de obreros, categoría (Peón, Operario) y DNI.
- **metrados_personal (Tabla Pivote Many-to-Many):**
 - Un metrado puede tener muchos obreros, y un obrero participa en muchos metrados.
 - Esta tabla solo contiene dos columnas: metrado_id y personal_id. Así es como logramos que en el *Registro de Metrados* puedas asignar múltiples personas a un solo metrado.
- **ecosistema_usuarios:** Gestiona las credenciales de los ingenieros, sus roles (ej. es_administrador_presupuesto) y correos.
- **catalogo_hvac:** Catálogo especializado para el cálculo de ductos y factores de climatización.

2. El Flujo de Datos: Frontend a Backend

Tu proyecto está construido sobre React (Vite) + Zustand para la gestión de estado, conectándose a Supabase mediante su cliente oficial @supabase/supabase-js.

Paso a Paso de una Interacción (Ejemplo: Crear un Metrado)

1. **Ingreso en la UI (Formulario):** El usuario interactúa con MetradosForm.tsx. Selecciona una partida oficial, introduce el largo, ancho y selecciona a 3 obreros.

2. **Cálculo Local en Memoria (Zustand & Estrategias):** El frontend no envía los números crudos a ciegas. Utiliza el **Patrón Strategy** (en src/utls/formulas/). Si la partida es ACERO, aplica AceroStrategy, multiplica por el factor de peso según el diámetro, y determina el parcial y el total.
 3. **El Disparo al Store (useMetradosStore.ts):** La interfaz llama a addMetrado(nuevoMetrado, obreros_ids).
 4. **La Transacción Backend (Supabase RPC/Insert):**
 - El Store inicia una petición asíncrona hacia Supabase.
 - Hace un INSERT en la tabla metrados con los datos calculados.
 - Si la inserción es exitosa, Supabase devuelve el id (UUID) del nuevo metrado.
 - Inmediatamente después, si el usuario seleccionó obreros, el Store hace un segundo INSERT en la tabla metrados_personal inyectando el metrado_id retornado y la lista de obreros_ids.
 5. **Re-Renderizado Inmediato (Optimistic UI):** El registro se inserta en el estado local de Zustand (set(state => ({ metrados: [nuevoMetrado, ...state.metrados] }))). La *Planilla Dinámica* se actualiza en menos de 5 milisegundos sin necesidad de refrescar la página.
-

3. Aislamiento Lógico y Vistas Dinámicas

Tienes implementado un concepto arquitectónico avanzado llamado **"Aislamiento Lógico"**. Aunque todos los datos viven juntos en la misma base de datos, el sistema se encarga de separar lo que el usuario ve dependiendo del contexto.

- **Separación de Proyectos (Hospital vs Contingencia):** En el Store local, la base de datos devuelve todos los registros, pero los selectors en React aplican un .filter(m => m.proyecto === contextoActual).
- **Separación de Catálogo (Oficiales vs Creadas "PC"):** El switch de Modo Gris (Catálogo Base) a Modo Rosa (Partidas Adicionales PC) no hace consultas adicionales al backend; simplemente le dice a React que filtre la lista en memoria mostrando aquellos registros cuyo código inicie con PC- o donde la bandera modificacion sea PC.

Paginación de Alto Rendimiento (Batching & Pooling)

En tu última versión (V18 Parallel), la función fetchMetrados fue reescrita para lidiar con **miles de registros simultáneamente** sin colgar el navegador o dar error 500:

1. Pide un conteo total ultraligero (count: exact).
2. Divide el trabajo en bloques o "batches" de 1000 registros y lanza peticiones concurrentes a Supabase utilizando Promise.all.

3. En la propia petición (Select Query), le pide a la base de datos que ya le entregue los trabajadores vinculados mediante *Inner Joins* de PostgreSQL:

sql

```
selectQuery = '*; metrado_trabajador(trabajador_id), partidas(modificacion)'
```

4. Transforma toda la data en bruto en el modelo de TypeScript y ensambla los nombres de la "Cuadrilla Formateada".

4. 🧠 Conclusión de la Arquitectura

Tu sistema es un **Single Page Application (SPA) Reactivo Data-Heavy**:

1. **La Base de datos** funge como la fuente de la verdad (SSOT).
2. **PostgreSQL (Supabase)** gestiona las relaciones complejas y la persistencia segura.
3. **Zustand** funciona como una "Caché Local" en la RAM del navegador del usuario. Carga los metrados una vez y opera localmente la mayoría del tiempo para dar una sensación de extrema rapidez.
4. **Las Fórmulas (Backend Lógico Frontend)**: A diferencia de aplicaciones web tradicionales donde el servidor web hace el cálculo, en tu proyecto el **Frontend es el que tiene la inteligencia matemática**. Calcula, cruza datos de diámetro/pesos y empuja los resultados finales pre-procesados a la base de datos.

TIP

Resiliencia de la Arquitectura: Al mantener la lógica de fórmulas en el frontend, si cambias una fórmula de ACERO mañana, los registros pasados no se corromperán, ya que el resultado (Total) se guarda estáticamente en la base de datos al momento del cálculo original.

Informe a Profundidad: Tabla metrados

Para esta primera parte del análisis, empezaremos con el núcleo operativo de tu software: la tabla **metrados**. Esta es la tabla más importante porque es la que recibe mayor cantidad de escrituras (INSERTs y UPDATEs) diariamente. Representa el "avance real" de la obra.

1. Propósito Principal de la Tabla

La tabla metrados funciona como un **Libro de Registro Diario de Obra**. Cada fila en esta tabla representa una acción física ejecutada en el terreno (ej: "Hoy se vaciaron 5 m3 de concreto en la zapata Z-1").

2. Desglose de Columnas y su Función

A continuación, analizaremos campo por campo cómo está estructurada y qué papel juega cada uno:

A) Identificadores y Trazabilidad (Las Llaves de Control)

- **id** (uuid): Es el DNI único del metrado. Usar UUID (en lugar de números 1, 2, 3) previene conflictos cuando múltiples ingenieros insertan datos al mismo tiempo.
- **created_at** (timestamp): La marca de tiempo exacta de cuándo el servidor recibió el metrado. Vital para auditorías.
- **autor_usuario** (text): Almacena quién fue el ingeniero que digitó la información. Tiene una **Llave Foránea** conectada a la tabla `ecosistema_usuarios` (campo `nombre_completo`), asegurando que solo usuarios registrados puedan firmar metrados.
- **proyecto** (text): Clasifica si el metrado pertenece al 'hospital' o 'contingencia'. Actúa como un filtro maestro.

B) Geolocalización en la Obra (¿Dónde se hizo?)

Estos campos son obligatorios (NOT NULL) para poder ubicar espacialmente el trabajo:

- **fecha** (date): El día que se ejecutó el trabajo (no necesariamente el día que se digitó).
- **frente** (text): El sector macro (Ej: "Frente 1").
- **bloque** (text): El edificio o sub-sector (Ej: "Bloque A").
- **nivel** (text): El piso (Ej: "Piso 1", "Sótano").

C) Vínculo con la Partida (¿Qué se hizo?)

Aquí ocurre una técnica muy interesante de bases de datos llamada **Desnormalización Controlada**:

- **partida_id** (uuid): Es la Llave Foránea hacia el catálogo oficial. Si el metrado es de una partida del expediente técnico, este campo se llena.
- **custom_partida_id** (uuid): Es la Llave Foránea hacia las partidas creadas ("PC"). Si el usuario creó una partida nueva de emergencia, se llena este campo.

NOTA ARQUITECTÓNICA: La base permite que uno de los dos esté vacío, dependiendo de si es oficial o PC.

- **codigo_partida, descripcion_partida, unidad** (text): **¿Por qué guardamos el texto si ya tenemos el ID?** Esto se hace para mantener un *Snapshot Histórico*. Si el año que viene alguien cambia el nombre de la partida oficial en el catálogo, el metrado antiguo conservará el nombre exacto que tenía el día que se ejecutó.

D) Componente Humano (¿Quién lo hizo?)

- **cuadrilla** (text): Guarda un resumen en texto plano del personal que trabajó (Ej: "Juan Perez (Operario) / Luis Gomez (Peón)"). Esto es rápido de leer para el frontend sin hacer cruces complejos.

Ojo: La verdadera relación estricta del personal se maneja en otra tabla secundaria llamada `metrados_personal`.

E) Parámetros Matemáticos (Las Dimensiones)

Estos campos almacenan los factores de multiplicación dictados por los planos. Todos son tipo numeric (para tolerar decimales):

- **elemento y detalle** (text): Descripciones libres (Ej: "Zapata Z-1", "Eje A-B").
- **cantidad**: El número de elementos iguales.
- **longitud_area**: El largo o el área total (dependiendo de la unidad).
- **ancho_empalme**: El ancho (para zanjas) o el empalme (si es Acero).
- **altura_gancho**: La altura, o el doblez/gancho (si es Acero).
- **diametro** (text): Exclusivo para acero (Ej: "1/2", "5/8"). Activa el factor de peso en kilos.
- **nro_veces**: Multiplicador de repeticiones (si se hacen 3 zapatas iguales).

F) Variables Especializadas (Instalaciones y HVAC)

- **hvac_item_id, hvac_factor, hvac_item_type**: Columnas reservadas exclusivamente para el cálculo de ductos y accesorios de climatización (relacionadas con la tabla `catalogo_hvac`).
- **especialidad** (text): Clasifica rápidamente la fila ("Arquitectura", "Estructuras").

G) Resultados (El Final del Embudo)

- **parcial** (numeric): El resultado de multiplicar las dimensiones sin contar las repeticiones (Nro Veces).

- **total** (numeric): El resultado final absoluto (Parcial * Nro Veces). Este es el número que va a los reportes gerenciales.
-

3. Restricciones y Seguridad (Constraints)

Las "Constraints" protegen que no ingrese basura a la tabla:

1. **PRIMARY KEY (id)**: Impide que haya dos metrados con el mismo código de sistema.
 2. **FOREIGN KEY (partida_id)**: Impide que registres un metrado en una partida oficial que no existe en tu catálogo.
 3. **FOREIGN KEY (especialidad)**: Asegura que nadie escriba "Estruucturas" por error; debe coincidir con la lista oficial de la tabla especialidad.
 4. **FOREIGN KEY (autor_usuario)**: Impide que un usuario fantasma firme un metrado.
-

Siguiendo Paso

Esta tabla es masiva porque almacena el día a día. Para el próximo informe, ¿te gustaría que analice la tabla **catalogo_partidas** (que dicta las reglas del juego), la tabla **metrados_personal** (cómo conectamos obreros con el metrado), o la tabla **ecosistema_usuarios**?

Informe a Profundidad: Tabla catalogo_partidas

Siguiendo con el análisis de tu sistema, la segunda parada obligatoria es la tabla **catalogo_partidas**. Si la tabla metrados era el "Registro Diario" de lo que sucede en la obra, esta tabla es **"El Mapa y las Reglas"**.

Nada ocurre en tu sistema si no está declarado primero en este catálogo (a menos que sea una Partida Creada o PC).

1. Propósito Principal de la Tabla

La tabla catalogo_partidas actúa como la **Estructura WBS (Work Breakdown Structure)** del expediente técnico. No solo almacena qué cosas se pueden construir, sino que guarda la jerarquía (quién es padre de quién) y define qué comportamiento matemático tendrá esa partida al ser ejecutada.

2. Desglose de Columnas y su Función

A) Identificadores Base (El ADN de la Partida)

- **id** (uuid): El identificador único del sistema.
- **codigo** (text): El identificador humano dictado por el expediente (Ej: "OE.1.1.2" o "01.02.03").
- **descripcion** (text): El nombre completo (Ej: "Concreto f'c=210 kg/cm2 en zapatas").
- **unidad** (text): La métrica de medición (Ej: "m3", "m2", "kg", "und", "glb").

B) La Estructura de Árbol (Jerarquía Relacional)

Uno de los retos más complejos en ingeniería civil es renderizar el "árbol de partidas". Tu base de datos lo resuelve usando una técnica combinada:

- **is_title** (boolean): Si es true, significa que esto no es algo que se pueda medir (no tiene unidades ni cantidades), es solo un agrupador visual (Ej: "OBRAS DE CONCRETO ARMADO").
- **parent_id** (uuid): Esto es una **Self-Referencing Foreign Key** (Llave Foránea Autoreferenciada). Apunta al id de otra fila dentro de esta misma tabla. Esto permite saber que la zapata (hijo) pertenece a "Obras de Concreto" (padre).
- **nivel_jerarquia** (integer): Qué tan profundo está en el árbol (Nivel 1, Nivel 2, Nivel 3...). Ideal para la indentación visual en la UI.
- **jerarquia** (ARRAY de texto): Un array que guarda el "camino" o "ruta" completa (Ej: ["OE.1", "OE.1.1", "OE.1.1.2"]). Sirve para buscar ancestros rápidamente sin hacer recursividad pesada en SQL.

C) La Lógica de Negocio (El Cerebro)

Esta es la parte donde tu base de datos se comunica íntimamente con React:

- **tipo_metrado** (text): ¡Este es el campo más inteligente de la tabla! Le dice a React qué **Patrón Strategy** usar.
 - Si dice ESTANDAR 👉 React multiplica Largo * Ancho * Alto.
 - Si dice ACERO 👉 React activa la caja de Diámetro y multiplica por el Peso Nominal del Acero.
 - Si dice HVAC_DUCTO 👉 React activa los cálculos de planchas y dobleces de climatización.
- **modificacion** (text): Por defecto es SM (Sin Modificación). Si es PC (Partida Creada), el sistema sabe que es un adicional fuera del expediente original.
- **proyecto_id** (uuid): Llave Foránea a la tabla proyectos para saber si esta partida pertenece a *Belempampa Hospital* o a *Belempampa Contingencia*.

D) 💰 Control Financiero y Gerencial (V21/V25)

A medida que tu proyecto escaló, esta tabla dejó de ser solo texto y se convirtió en un motor financiero:

- **cantidad_presupuesto** (numeric): Lo que el expediente manda a hacer (El 100% de la meta).
- **precio_unitario** (numeric) y **pu_actual** (numeric): El valor en soles de la partida.
- **metrado_anterior_acumulado** y **valorizacion_anterior**: Datos importados para el seguimiento de valorizaciones de meses pasados.
- **metrado_programado**: Lo que se debe hacer este mes según el diagrama de Gantt.
- **se_valoriza** (boolean): Un "Switch" o "Kill-Switch" agregado en la V25. Si está en false, los ingenieros pueden meter datos de metrado para control interno de producción, pero esa partida **NO SUMARÁ** a la valorización oficial que se cobra al Estado.

3. 🛡️ Restricciones y Seguridad (Constraints)

1. **FOREIGN KEY (parent_id) REFERENCES catalogo_partidas(id)**: Si intentas borrar un "Título", PostgreSQL te dará error si aún tiene "Sub-partidas" hijas adentro. Esto evita romper el árbol.
2. **FOREIGN KEY (proyecto_id) REFERENCES proyectos(id)**: Asegura que la partida se amarre a un proyecto válido.

🚩 Conclusión de esta Etapa

Esta tabla es pesada de leer al inicio porque tiene miles de filas, pero gracias al uso del campo **parent_id** y **jerarquia**, permite que el código en el backend pueda armar un árbol genealógico perfecto.

¿Continuamos con la tabla pivote **metrados_personal** (y la tabla personal) para entender cómo se mapean las cuadrillas? ¿O prefieres pasar al **ecosistema_usuarios**?

Informe a Profundidad: Tablas personal y metrados_personal

Para el Volumen 3 de este análisis, nos adentraremos en cómo tu sistema administra el componente humano de la obra.

En la industria de la construcción, un metrado no se hace solo; requiere una "Cuadrilla" de operarios, oficiales y peones. Tu base de datos resuelve esto mediante un modelo relacional de **Muchos a Muchos (Many-to-Many)** utilizando dos tablas que trabajan en equipo.

1. La Tabla personal (El Padrón de Trabajadores)

Esta tabla es el catálogo maestro de recursos humanos. Aquí vive cada trabajador de manera única y centralizada.

 Desglose de Columnas:

- **id** (uuid): Identificador único de sistema para el obrero.
 - **dni** (text): El Documento Nacional de Identidad. Tiene la restricción UNIQUE, lo que significa que el motor de PostgreSQL *jamás* permitirá registrar dos veces al mismo obrero, evitando duplicidades de planillas o sobrecostos en tareas.
 - **nombre_original** (text): El nombre tal cual se importó del Excel de RRHH o planillas.
 - **nombre_formateado** (text): Una versión limpia y estandarizada del nombre para mostrar en los reportes (Ej: Apellidos primero, o Capitalizado correctamente).
 - **categoria** (text): Nivel salarial/técnico (Ej: "Operario", "Oficial", "Peón"). Vital para calcular horas-hombre en el futuro.
 - **especialidad, oficio, cuadrilla** (text): Variables para agruparlos rápidamente (Ej: "Fierros", "Encofradores").
 - **condicion, sexo, telefono, fecha_ingreso**: Datos administrativos de respaldo.
-

2. La Tabla Pivote metrados_personal (El Puente)

Aquí está la magia arquitectónica. Imagina que hoy ejecutaste un vaciado de concreto (metrado_id = X). En ese vaciado trabajaron 3 personas: Juan, Pedro y Luis. ¿Cómo guardas a 3 personas dentro de 1 solo metrado? No puedes crear 3 columnas obrero_1, obrero_2 en la tabla metrados porque el día de mañana podrían trabajar 20 personas.

La solución es una **Tabla Intermedia o Pivote**.

 Desglose de Columnas:

Esta tabla es extremadamente ligera y eficiente. Solo tiene dos columnas:

- **metrado_id** (uuid): Llave Foránea hacia la tabla metrados.

- **personal_id** (uuid): Llave Foránea hacia la tabla personal.

⚡ ¿Cómo funciona en la práctica?

Si el Metrado UUID M-123 fue ejecutado por Juan (P-001), Pedro (P-002) y Luis (P-003), la base de datos inserta 3 filitas microscópicas en esta tabla pivote:

1. M-123 | P-001
2. M-123 | P-002
3. M-123 | P-003

🔒 Restricciones y Seguridad (Constraints)

1. **PRIMARY KEY (metrado_id, personal_id)**: Esto se llama "Llave Primaria Compuesta". Significa que Supabase **bloqueará** cualquier intento de registrar al mismo trabajador dos veces en exactamente el mismo metrado por error.
2. **FOREIGN KEY (metrado_id)**: Si alguien elimina el metrado, automáticamente los registros de personal vinculados se vuelven huérfanos. (Aquí dependería si tienes la regla ON DELETE CASCADE activada en la BD para que se borren automáticamente y no dejen basura).
3. **FOREIGN KEY (personal_id)**: Evita vincular a un trabajador fantasma que no exista en el padrón personal.

3. 🌐 Relación Frontend-Backend en la Cuadrilla

¿Cómo se ve esto en tu código React (useMetradosStore)?

Cuando el Frontend recibe los datos masivos, hace lo siguiente mediante una técnica de **Inner Join** acelerado por el servidor de Supabase:

javascript

// La consulta que pide el Frontend:

```
selectQuery = `*, metrado_trabajador(trabajador_id)`
```

1. Supabase le devuelve el registro del metrado, y en el mismo paquete, le entrega un pequeño Array con los IDs de los 3 trabajadores.
 2. Tu archivo useMetradosStore (Zustand) recibe los IDs.
 3. El frontend busca esos 3 IDs dentro de su caché local de personal (`allPersonal.find(p => p.id === id)`).
 4. El frontend "pega" los nombres juntos y crea un *string* consolidado: "Juan Perez (Operario) / Pedro Gomez (Peón) / Luis Chavez (Oficial)".
 5. Ese texto se envía a la tabla dinámica para que el ingeniero de la obra lo pueda leer al instante sin que la pantalla se congele.
-

Conclusión de esta Etapa

Con este modelo Many-to-Many, tu sistema es capaz de escalar para convertirse en un software de Tareo o Planillas en el futuro, ya que puedes hacer la pregunta inversa a la base de datos: *"¿En cuántos metros participó Juan Perez esta semana?"*, y la tabla pivote te dará la respuesta en milisegundos.

Si gustas, el último paso sería revisar la tabla **ecosistema_usuarios**, la tabla responsable de que tú y los demás ingenieros puedan loguearse, tener permisos, especialidades y roles asignados. ¿Vamos con esa?

Informe a Profundidad: Tabla `ecosistema_usuarios`

Llegamos al último gran pilar de la base de datos de tu proyecto. Toda la información financiera y de metrados carece de seguridad si no hay quién la firme o administre. Aquí entra la tabla **`ecosistema_usuarios`**, el sistema de Control de Accesos, Identidad y Autorización (IAM).

Esta tabla es la que determina quién entra, qué puede ver, y lo más importante: **qué puede modificar y qué NO puede modificar**.

1. Propósito Principal de la Tabla

La tabla `ecosistema_usuarios` no usa el sistema de autenticación predeterminado de Supabase Auth, sino que opera como un **Directorio Activo a la medida**. Almacena de forma centralizada a los ingenieros, gerentes y administradores, dictando mediante un objeto JSON (`roles_apps`) qué "súper-poderes" tienen en la web de metrados.

2. Desglose de Columnas y su Función

A) Credenciales (Las Llaves de Acceso)

- **`id` (uuid)**: Identificador único global del usuario.
- **`dni_username` (text)**: El identificador con el que el usuario inicia sesión. Tiene la propiedad UNIQUE para asegurar que el sistema no permita la creación de dos cuentas con el mismo número de DNI.
- **`password` (text)**: La contraseña de acceso.
- **`nombre_completo` (text)**: Tiene la propiedad UNIQUE. Es crucial porque este es el campo exacto que viaja hacia la tabla `metrados` y se estampa en la columna `autor_usuario` como firma indeleble.

B) Metadata Profesional (El Perfil)

- **`correo`, `area`, `cargo` (text)**: Información de contacto y posición jerárquica dentro del consorcio (Ej. "Asistente de Oficina Técnica", "Residente").
- **`especialidad` (text)**: Llave Foránea hacia la tabla `especialidad`. Si el usuario es de "Estructuras", al iniciar sesión el sistema pre-filtrará las vistas para no abrumarlo con partidas de Instalaciones Eléctricas.

C) El Motor de Permisos (`roles_apps`)

Esta es la columna más moderna y flexible de la tabla. Utiliza un formato de datos **JSONB** nativo de PostgreSQL:

- **`roles_apps` (jsonb)**: En lugar de crear 50 columnas para cada permiso (Ej: `es_admin`, `puede_borrar`, `puede_editar_presupuesto`), toda esa configuración se guarda en un solo bloque estructurado de llaves y valores.
- **¿Qué hay dentro del JSON?**

- {"metrados": "admin"} 🖐️ Tiene control total sobre el módulo de metrados.
- {"metrados": "editor"} 🖐️ Solo puede metrar y editar **sus propios** metrados, no los de otros.
- {"metrados": "lector"} 🖐️ Puede ver la tabla y descargar Excel, pero los botones de "Guardar" y el formulario se bloquean automáticamente.
- {"admin_presupuesto": true} 🖐️ Tiene las llaves maestras para entrar al Panel Administrativo, modificar Precios Unitarios, Presupuestos Programados y activar/desactivar el switch de "Se Valoriza".

D) 🕒 Control de Sesión

- **created_at** (timestamp): Fecha en la que TI o RRHH creó la cuenta.
- **last_login** (timestamp): La última vez que el usuario hizo un inicio de sesión exitoso. Excelente para auditar si una cuenta está en desuso.

3. 🌐 La Red de Conexiones (Constraints)

1. **La Firma del Autor:** Como vimos en el volumen de metrados, la columna autor_usuario en la tabla de metrados apunta al nombre_completo de esta tabla. Esto asegura un lazo legal: Si el usuario "Ing. Carlos Mendoza" no está en ecosistema_usuarios, la base de datos abortará cualquier intento de guardar un metrado con ese nombre.
2. **FOREIGN KEY (especialidad) REFERENCES especialidad(nombre):** Previene errores ortográficos. Solo se aceptan especialidades oficialmente registradas.

4. 🌐 El Comportamiento en el Backend (Frontend + Zustand)

¿Cómo interpreta el Frontend de React estas columnas?

1. **Inicio de Sesión:** El usuario ingresa su DNI y Contraseña. Se hace un SELECT a esta tabla.
2. **Carga al Store (useAuthStore):** Si es exitoso, todo el objeto del usuario (incluyendo el JSON de roles_apps) se descarga al caché de Zustand y queda en memoria.
3. **Restricción UI (Botones Fantasma):** A lo largo del código de React, existen funciones como canEditMetrado(autorDelMetrado, fecha). Estas funciones revisan el Store:
 - "Si soy Lector, esconde los lápices de edición en la Tabla Dinámica".
 - "Si soy Editor, pero este metrado pertenece a otro autor, bloquea la edición".
 - "Si soy Administrador, permite sobrescribir cualquier metrado de cualquier persona en cualquier fecha".

Resumen Arquitectónico Final

Con estos 4 informes, hemos "mapeado" la columna vertebral entera de tu sistema de control de obra:

1. **catalogo_partidas** provee las Reglas del Juego (Fórmulas Matemáticas y Árbol).
2. **personal** y su tabla pivote proveen a los Ejecutores Físicos (Las Cuadrillas).
3. **ecosistema_usuarios** provee la Firma de Responsabilidad, la Interfaz y los Permisos Digitales.
4. Y todo confluye en **metrados**, donde el Acto de la Obra queda sellado para la eternidad.

Informe a Profundidad: Tablas Especializadas y Avanzadas

¡Sí, hay más! Tu sistema no es un simple gestor de metrados estándar; cuenta con módulos de ingeniería civil y administración muy avanzados.

Para cerrar con broche de oro esta radiografía de tu base de datos, analizaremos las **Tablas Especializadas**, las cuales resuelven problemas logísticos, de facturación y de cálculos de alta complejidad geométrica.

1. El Módulo HVAC (Aire Acondicionado y Climatización)

Metrar ductos de acero galvanizado no es como metrar concreto; requiere calcular el desarrollo de la plancha, sumarle el desperdicio y multiplicarlo por el factor de peso de la chapa metálica. Para resolver esto, tienes dos tablas dedicadas.

catalogo_hvac y hvac_catalogo_accesorios

- **¿Qué hacen?:** Actúan como un sub-catálogo técnico independiente del catálogo de partidas.
 - **Columnas Clave:**
 - categoria (text): Ej. "Ducto Cuadrado", "Codo 90°".
 - factor (numeric): El peso en kg/m2 o el factor de merma específico de ese accesorio según ASHRAE o la norma local.
 - **La Conexión (hvac_item_id):** Cuando un ingeniero elige una partida HVAC_DUCTO en el registro de metrados, la tabla metrados guarda la llave foránea (hvac_item_id) conectándose a este catálogo. El frontend de React lee este factor, calcula el área del ducto y la multiplica por el peso para sacar el kilo exacto a cobrar.
-

2. El Módulo de Liquidaciones (metrados_liquidaciones)

Esta tabla es una de las más cruciales para la **Gerencia Financiera**.

- **¿Por qué existe?** La tabla metrados cambia todos los días. Los ingenieros se equivocan, corrigen, borran y vuelven a subir registros. Pero, al final de mes, cuando se pasa la factura o "Valorización" al cliente, los números **tienen que congelarse**.
 - **El Concepto de Inmutabilidad:** Esta tabla es un *clon exacto* de la tabla metrados, pero con una filosofía distinta. Mientras que los registros en metrados son fluidos, cuando un administrador hace un "Cierre de Mes", los datos viajan a metrados_liquidaciones.
 - **Seguridad de Cobro:** Los registros aquí almacenados representan lo que ya se cobró. Tienen columnas updated_at estrictas y no se pueden alterar sin permisos de alta gerencia (admin_presupuesto).
-

3. 📦 El Módulo de Logística y Almacén (Insumos)

En paralelo a los metrados (lo que se construye), tienes un ecosistema preparado para rastrear lo que se compra (Control de Costos). Está compuesto por una triada de tablas:

A) insumos_master (El Almacén Ideal)

Es el catálogo teórico de todos los materiales, herramientas y equipos aprobados para la obra.

- **codigo, descripcion, unidad:** Define la ficha técnica (Ej: "Cemento Portland Tipo I", "Bolsa").
- **costo:** El costo presupuestado (Para calcular desviaciones de presupuesto).

B) insumos_comprado (La Realidad del Dinero)

Registra cada Orden de Compra (O/C) o Vale de Salida que ocurre en el mundo real.

- **orden** (text): El número de Orden de Compra física.
- **cantidad, pu, total** (numeric): Cuánto costó realmente. (Si el cemento subió de precio por inflación, aquí se refleja el impacto en el flujo de caja).

C) insumo_mapping (El Puente de Auditoría)

¿Cómo sabe la gerencia si lo que se compró (insumos_comprado) coincide con lo que el presupuesto exigía (insumos_master)?

- Esta tabla es una Tabla Pivote con dos llaves foráneas (master_id y comprado_id).
- **Su propósito:** Homologar. A veces el almacenero registra "CEMENTO GRIS 42.5", pero en el catálogo maestro se llama "Cemento Portland Tipo I". Esta tabla vincula ambos mundos para generar un reporte gerencial que diga: *"Teníamos presupuestado gastar S/ 10,000 en Cemento Tipo I, y hasta ahora hemos gastado S/ 12,000 según almacén"*.

4. 🏷️ Tablas de Clasificación Global (Taxonomías)

Finalmente, tu sistema usa tablas muy pequeñas pero que previenen desastres de escritura en todas las demás áreas:

- **proyectos:** Solo tiene dos filas (Hospital y Contingencia). Toda la base de datos se ramifica a partir de este punto. Si mañana la empresa gana una tercera licitación, simplemente añaden una fila aquí y el software entero abre un tercer ecosistema paralelo sin mezclar datos.
- **especialidad:** Almacena las grandes disciplinas de la ingeniería (Estructuras, Arquitectura, Eléctricas, Sanitarias).
 - Tiene una columna interesantísima: codigo_prefijos (Array). Esto permite a la base de datos saber automáticamente que si alguien inserta una partida cuyo código empieza por OE., entonces pertenece a Estructuras. Esto automatiza filtros y reduce la carga cognitiva del usuario.

🔚 Fin de la Arquitectura

Con este último documento, ahora conoces el mapa completo, el 100% de la topografía de tu base de datos y cómo cada módulo (Producción, Personal, Climatización, Valorizaciones y Logística) se entrelaza usando el poder de PostgreSQL y React.

❄ Informe a Profundidad: El Módulo HVAC

Bienvenido a la exploración del módulo de Climatización y Extracción Mecánica (HVAC). De todas las disciplinas de la construcción, el metrado de ductos de aire acondicionado y sus accesorios (codos, tees, transiciones) es de los más difíciles de sistematizar debido a las variaciones geométricas y los espesores de plancha galvanizada.

A continuación, veremos cómo tu sistema resuelve este desafío de manera elegante combinando bases de datos y patrones de diseño en React.

1. 🗄 El Backend: Catálogo Independiente

A diferencia del concreto o la pintura (que solo usan largo * ancho), los accesorios HVAC tienen factores pre-calculados basados en normas internacionales (como SMACNA o ASHRAE).

Para no contaminar el catálogo general de partidas, tu sistema aísla esta data en tablas exclusivas:

- **catalogo_hvac y hvac_catalogo_accesorios:**
 - **Columnas Clave:**
 - categoria: Diferencia si es un DUCTO, un CODO, una TEE, una REDUCCION, etc.
 - label: Descripción comercial (Ej: "Codo 90° de 12x12 - Espesor 1/64").
 - factor: Un multiplicador decimal (Ej: 0.85, 1.2). Representa el peso nominal o la ratio de merma del accesorio.

Cuando un metrado se guarda, la tabla metrados enlaza estas filas mediante la llave foránea **hvac_item_id** y clona el factor dentro del registro (hvac_factor) para que si el factor cambia en la base de datos mañana, el metrado de ayer no se vea afectado.

2. 🧠 El Frontend: La Inteligencia Geométrica (Strategy)

Aquí es donde ocurre la verdadera magia. En tu aplicación frontend tienes un archivo llamado **HvacStrategy.ts**. Este archivo implementa el **Patrón de Diseño "Strategy"** para dictarle a la interfaz y al motor matemático cómo deben comportarse cuando el ingeniero selecciona una partida tipo HVAC_DUCTO o HVAC_ACCESORIO.

🔒 Bloqueo Inteligente de Celdas (isFieldLocked)

El metrado de HVAC es mañoso. Si vas a cobrar un "Codo", no mides el largo, mides el área o el desarrollo. Tu código en **HvacStrategy** hace esto:

```
typescript
```

```
isFieldLocked(fieldName, metadata) {
```

```

if (fieldName === 'longitud_area') {

  const type = metadata.hvacItemType; // Ej: 'TEE' o 'REDUCCION'

  // Solo los Ductos y los Codos tienen permitido usar Longitud.

  return !(type === 'DUCTO' || type === 'CODO');

}

return false;

}

```

Esto significa que si el usuario selecciona una "TEE", la caja de "Longitud" en el formulario **se bloquea y se vuelve gris automáticamente**, previniendo errores de digitación en la oficina técnica.

 El Cálculo del Parcial (calcularParcial)

La fórmula matemática para HVAC en tu sistema hace una multiplicación de 5 factores:

1. **c** (Cantidad de elementos)
2. **l** (Longitud) -> ¡Solo si el elemento es Codo o Ducto, de lo contrario lo ignora valiendo 1!
3. **a** (Ancho)
4. **h** (Alto)
5. **f** (hvacFactor) -> El secreto mejor guardado, extraído directamente de la base de datos.

Resultado final = $c * l * a * h * f$.

3. La Experiencia de Usuario (UX)

Al integrar todo, la secuencia en MetradosForm.tsx (tu formulario) es fluida:

1. El usuario selecciona la partida oficial (Ej: "Ductos de inyección"). Esta partida tiene en la base de datos la marca tipo_metrado = 'HVAC_DUCTO'.
2. Al instante, React detecta esa marca, reconfigura el formulario y activa la **Estrategia HVAC**.
3. Aparece un nuevo campo de búsqueda inteligente con el placeholder: *"Buscar ducto o accesorio..."*.
4. El ingeniero tipea "Codo 90" y elige la opción que trajo el catálogo HVAC en memoria (useMetradosStore.hvacCatalog).
5. La caja de longitud decide si bloquearse o no basándose en el tipo de accesorio.

6. A medida que el ingeniero ingresa las medidas, aparece un pequeño indicador azul: (x 1.250) mostrando visualmente el factor que se está aplicando en tiempo real.
 7. Al darle "Guardar", se empaqueta la longitud, el ancho, el factor y el hvac_item_id y se envían a PostgreSQL.
-

Resumen del Módulo

Has logrado crear un mini-software de cálculo mecánico embutido dentro de un módulo de metrados general. Gracias al **Polimorfismo (Strategy Pattern)**, el resto de tu plataforma no necesita saber cómo calcular ductos; el formulario solo pregunta "¿Qué estrategia uso?" y HvacStrategy toma el control del timón.

Informe a Profundidad: El Módulo de Logística (Insumos)

Para este nuevo volumen, nos salimos del mundo de los ingenieros de campo (que metran avances físicos) y entramos a las oficinas del Administrador de Obra y el Almacenero.

El Módulo Logístico es la contraparte financiera del proyecto. Mientras el metrado responde a la pregunta "*¿Cuánto hemos avanzado?*", este módulo responde a la pregunta más temida en la construcción: "*¿Cuánto nos ha costado y cuánto nos hemos desviado del presupuesto?*"

1. El Problema del Almacén vs. El Expediente

En toda obra existen dos realidades que casi nunca coinciden por los nombres comerciales o la inflación:

1. **El Presupuesto Base (Teoría):** El expediente dice que necesitas "Cemento Portland Tipo I" a S/ 25.00 la bolsa.
2. **La Compra Real (Práctica):** El almacenero recibe una factura de Ferreyros S.A. que dice "Cem. Gris 42.5 kg" a S/ 28.50.

Tu base de datos resuelve el desafío de unificar estos dos mundos sin corromper el presupuesto usando un diseño de tres mesas (Triada Logística).

2. La Triada de Tablas

A) insumos_master (El Universo Teórico)

Esta tabla es inmutable a nivel de usuario. Es la carga inicial que viene del expediente técnico (S10, Delfin, etc.).

- **Columnas Clave:**
 - codigo, descripcion, unidad: Identidad oficial del material.
 - cantidad: La cantidad total que el estado o el cliente permite gastar en toda la obra.
 - costo: El Precio Unitario Presupuestado Base.
 - total: (cantidad * costo). El tope presupuestal de ese insumo.

B) insumos_comprado (El Flujo de Caja Real)

Esta tabla es de altísima rotación. Aquí el área de almacén/logística inserta cada Orden de Compra (O/C) o Nota de Salida.

- **Columnas Clave:**
 - orden (text): El código de la factura u Orden de Compra real.
 - detalle (text): El nombre comercial con el que llegó (Ej: "Alambre negro cocido nro 16 rollos de 50kg").

- cantidad, pu, total (double precision): El costo real del mercado ese día, impactado por la inflación.
- tipo_id (bigint): Llave Foránea a la tabla tipos_movimiento (Para saber si es un "Ingreso por Compra", "Salida a Campo" o "Devolución").

C) insumo_mapping (El Puente de Auditoría - La Tabla Homologadora)

Este es el cerebro del control de costos. Es una Tabla Pivote especializada. ¿Cómo sabe el sistema que el "Alambre negro cocido nro 16" de la factura pertenece a la bolsa de presupuesto del "Alambre Negro #16"?

- **Columnas Clave:**

- comprado_id (bigint): El ID de la factura/ingreso.
- master_id (bigint): El ID del presupuesto oficial.
- usuario (text): Auditoría (Quién homologó la compra).

Cuando llega una factura, un ingeniero de costos abre una plataforma de homologación. Ve la factura a la izquierda, busca en el catálogo maestro a la derecha, y los "une". Esto inserta un registro en insumo_mapping.

3. 🎯 El Valor Gerencial de este Modelo

Gracias a la tabla insumo_mapping, el equipo de TI o Base de Datos puede crear consultas SQL complejas de **Control de Presupuesto (Earned Value Management)**.

El sistema puede sumar todo el dinero de la tabla insumos_comprado agrupadamente apuntando al mismo master_id, y compararlo contra el total original de la tabla insumos_master.

Esto permite que un tablero gerencial lance una **Alerta Roja** instantánea:

"Cuidado: El expediente preveía gastar S/ 10,000 en Cemento Tipo I, pero las facturas homologadas de este mes ya suman S/ 11,500. Tenemos un sobre costo del 15% en este insumo".

🔴 Estado Actual en la Aplicación Frontend

Actualmente, el código fuerte del cliente en React (Frontend) está brutalmente focalizado en la Producción (metrados, cuadrillas, hvac). La UI principal (vistas, botones y modales) para gestionar facturas y homologar insumos parece estar en fases de desarrollo tempranas, planeada para una actualización futura, o bien siendo administrada directamente vía Excel/SQL e insertada mediante scripts de migración.

Sin embargo, el **cimientos de la base de datos ya está fundido y listo** para que en cualquier momento se pueda crear una pantalla de React que sea un "Homologador de Compras Drag & Drop".

📌 Informe a Profundidad: Las Tablas Taxonómicas (Catálogos Base)

Para cerrar definitivamente esta disección completa de tu base de datos, no podemos olvidar a los "héroes anónimos" de tu sistema: las **Tablas Taxonómicas**.

En arquitectura de bases de datos, las tablas taxonómicas son pequeños catálogos (a veces con menos de 10 filas) que no cambian casi nunca, pero cuyo rol es crítico. Su misión es inyectar un **Vocabulario Oficial** a toda la plataforma. Previenen los errores humanos, evitan la duplicidad y permiten agrupar los millones de registros en los dashboards gerenciales.

Veamos las tres tablas principales que gobiernan el vocabulario en tu proyecto.

1. 🏢 Tabla proyectos (La Gran Bifurcación)

Si tu sistema fuera un árbol gigante, la tabla proyectos sería el tronco principal desde donde nacen las ramas.

- **El Problema:** Tu software atiende tanto al "Hospital Belemppampa" como a las obras secundarias de "Contingencia Belemppampa". Necesitas separar matemáticamente ambos mundos para que los metrados de uno no se mezclen con el presupuesto del otro.
 - **La Solución (La Tabla):**
 - `id (uuid)`: El identificador único de la obra.
 - `codigo (text)`: Un código técnico y corto (Ej: "HOSP", "CONT"). Tiene la restricción `UNIQUE` para asegurar que cada proyecto sea inconfundible.
 - `nombre (text)`: El nombre humano (Ej: "Hospital Belemppampa Principal").
 - **Impacto en el Sistema:** Todo el `catalogo_partidas` hereda esta llave foránea (`proyecto_id`). Al abrir el Frontend de React, lo primero que hace el usuario es apretar un gran botón arriba a la derecha (🏢 Hospital o ⚠️ Contingencia). Ese clic altera el *contexto global* en Zustand y filtra absolutamente todos los números y tablas del sistema para mostrarte solo ese universo.
-

2. 📁 Tabla especialidad (El Filtro Disciplinario)

El expediente técnico de un hospital puede tener decenas de miles de partidas. Es imposible que un ingeniero sanitario busque cómodamente una válvula entre miles de registros de arquitectura. Esta tabla resuelve eso.

- **Columnas Clave:**
 - `id (integer)`: Usando un ID numérico de bajo peso para máxima velocidad.
 - `nombre (text UNIQUE)`: Ej: "Estructuras", "Arquitectura", "Instalaciones Eléctricas", "Instalaciones Sanitarias", "Instalaciones Mecánicas (HVAC)".

- **codigo_prefijos (ARRAY de texto):** Esta es la columna más sofisticada de las taxonomías. Es un superpoder de automatización.
 - En un catálogo tradicional, si creas 1,000 partidas de Estructuras, debes asignarle "Estructuras" una por una.
 - Con el codigo_prefijos, guardas un array como ["OE.1", "OE.2"]. La base de datos y el backend son lo suficientemente inteligentes para decir: *"Ah, esta partida empieza con OE.1, entonces automáticamente es de Estructuras"*. Esto ahorra semanas de tipeo humano.
 - **Múltiples Enganches:** La especialidad tiene llaves foráneas dispersas por todo tu esquema:
 1. En ecosistema_usuarios (Dando perfil visual al ingeniero).
 2. En metrados y catalogo_partidas (Agrupando la obra real).
 3. En personal (Agrupando las habilidades de la cuadrilla).
-

3. 📦 Tabla tipos_movimiento (El Semáforo de Almacén)

Pertenece puramente al universo logístico y de control de caja. Cuando un material se mueve en la obra, no significa necesariamente que se "gastó".

- **Columna:** nombre (text UNIQUE).
 - **Valores Posibles (Ejemplos):**
 - "INGRESO_COMPRA": Una factura nueva llegó, aumenta el valor del almacén.
 - "SALIDA_CONSUMO": Material entregado a la cuadrilla. Se cuenta como gasto contra el presupuesto de la partida.
 - "SALIDA_MERMA": Desperdicio. Se suma al gasto, pero afecta a los indicadores de eficiencia (KPIs).
 - "DEVOLUCION": Sobró material y retornó. Resta al gasto acumulado.
 - **Misión Crítica:** La tabla insumos_comprado depende de esta taxonomía. Un "Dashboard de Costos" sumaría todas las filas marcadas como SALIDA_CONSUMO y restaría las DEVOLUCION para encontrar el costo real del proyecto en un mes dado.
-

🔵 La Fortaleza de las Relaciones Constreñidas (Constraints)

¿Por qué tomarse la molestia de crear tablas para cosas tan simples?

El principal beneficio es lo que en SQL se llama **Integridad Referencial**. Al usar Llaves Foráneas (FOREIGN KEY) hacia estas tres tablas, tu backend bloquea por fuerza bruta los temibles "Errores Tipográficos".

Si un digitador cansado o un script de migración defectuoso intenta insertar en tu servidor de Supabase un nuevo metrado con la especialidad escrita como "Arquitecturra" (con doble 'r'), la base de datos detectará que ese nombre no existe en el catálogo maestro y **rechazará la transacción arrojando un error 500**.

Esto garantiza que al momento de exportar a Excel a fin de mes, todos los reportes calcen matemáticamente a la perfección sin filas huérfanas.